
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 1

Introdução aos Computadores, à Internet e à Web

Exercícios de Autorrevisão

Respostas completamente explicadas, passo a passo,
no estilo incremental Deitel

Nota ao Estudante

Bem-vindo ao primeiro conjunto de exercícios de autorrevisão do livro *C: Como Programar*. Antes de prosseguir, queremos ressaltar a importância desta seção. Os exercícios de autorrevisão são concebidos para que você possa verificar, de forma imediata e autônoma, se os conceitos fundamentais do capítulo foram assimilados. Cada questão é respondida de maneira detalhada, com explicações que vão além do simples gabarito, sempre conectando a resposta ao conteúdo desenvolvido no Capítulo 1.

Boa prática de programação

Leia cada enunciado com atenção antes de consultar a resposta. Tente responder de memória. Essa prática ativa de recuperação de conteúdo é uma das técnicas mais eficazes de aprendizado comprovadas pela ciência cognitiva.

Contents

1	Exercício 1.1 – Preenchimento de Lacunas: Conceitos Gerais	3
2	Exercício 1.2 – Preenchimento de Lacunas: Ambiente de Desenvolvimento em C	8

1. Exercício 1.1 – Preenchimento de Lacunas: Conceitos Gerais

Enunciado

Preencha os espaços em cada uma das seguintes sentenças:

- a) A empresa que popularizou a computação pessoal foi _____.
- b) O computador que tornou a computação pessoal possível no comércio e na indústria foi o _____.
- c) Computadores processam dados sob o controle de conjuntos de instruções chamadas _____ de computador.
- d) As seis unidades lógicas do computador são _____, _____, _____, _____, _____ e _____.
- e) Os três tipos de linguagem que discutimos são _____, _____ e _____.
- f) Os programas que traduzem programas em linguagem de alto nível na linguagem de máquina são chamados de _____.
- g) C é bastante conhecida como a linguagem de desenvolvimento do sistema operacional _____.
- h) O Department of Defense desenvolveu a linguagem Ada com uma capacidade chamada de _____, que permite que os programadores especifiquem as atividades que podem ocorrer em paralelo.

Item (a) – A empresa que popularizou a computação pessoal

Resposta detalhada

Resposta: Apple

Explicação passo a passo:

Na Seção 1.4 do Capítulo 1 aprendemos que, em 1977, a **Apple Computer** popularizou a computação pessoal. Antes disso, os computadores eram equipamentos de grande porte, caros e restritos a organizações governamentais, acadêmicas e grandes empresas. A Apple lançou computadores pessoais acessíveis, abrindo as portas para que indivíduos pudessem usar computadores em suas casas e pequenos negócios.

É importante não confundir “popularizar” com “introduzir”. A IBM introduziu o *IBM Personal Computer* (PC) em 1981 (item b), mas quem *popularizou* – isto é, tornou a computação pessoal amplamente acessível e desejada – foi a Apple, quatro anos antes.

Item (b) – O computador que levou a computação pessoal ao comércio e à indústria**Resposta detalhada**

Resposta: IBM Personal Computer (IBM PC)

Explicação passo a passo:

Em 1981, a IBM – na época o maior vendedor de computadores do mundo – lançou o **IBM Personal Computer**. Esse evento foi decisivo: pelo fato de a IBM já estar consolidada em empresas, governos e indústrias (por meio de seus mainframes), a chegada do IBM PC deu credibilidade comercial à computação pessoal. Empresas passaram a adotar computadores pessoais em larga escala, transformando para sempre os ambientes de trabalho.

Note a distinção histórica: a Apple democratizou o uso *pessoal*; o IBM PC democratizou o uso *corporativo*. Ambos, juntos, definiram a era da computação pessoal.

Item (c) – Conjuntos de instruções que controlam os computadores**Resposta detalhada**

Resposta: programas

Explicação passo a passo:

Da Seção 1.2: “*Computadores processam dados sob o controle de conjuntos de instruções chamados **programas** de computador.*”

Essa definição é fundamental. Um computador, por si só, é apenas hardware – circuitos, chips e dispositivos físicos. O que dá vida a esse hardware e o faz executar tarefas úteis são os **programas** (software) escritos por programadores. Programar em C é exatamente a arte de criar esses conjuntos de instruções ordenadas.

Item (d) – As seis unidades lógicas do computador**Resposta detalhada**

Resposta: (1) unidade de entrada, (2) unidade de saída, (3) unidade de memória, (4) unidade lógica e aritmética (ALU), (5) unidade central de processamento (CPU), (6) unidade de armazenamento secundário

Explicação passo a passo:

A Seção 1.3 descreve que, independentemente de sua aparência física, todo computador pode ser dividido em **seis unidades lógicas**. Veja cada uma delas:

- **Unidade de entrada:** A seção “receptora” do computador. Recebe dados e programas do mundo externo (teclado, mouse, sensores, etc.) e

os disponibiliza para as demais unidades.

- **Unidade de saída:** A seção de “expedição”. Pega as informações processadas e as envia para dispositivos externos (monitor, impressora, alto-falantes, etc.).
- **Unidade de memória:** O “depósito rápido” do computador. Armazena dados e programas temporariamente enquanto estão em uso. Suas informações são *voláteis* (perdidas quando o computador é desligado). Também chamada de *memória primária* ou *RAM*.
- **Unidade lógica e aritmética (ALU):** A seção de “processamento”. Realiza cálculos matemáticos (adição, subtração, multiplicação, divisão) e tomada de decisões lógicas (comparações entre valores).
- **Unidade central de processamento (CPU):** A seção “administrativa”. Coordena e supervisiona todas as demais unidades. É ela que diz *quando* a unidade de entrada deve ler dados, *quando* a ALU deve calcular, *quando* a unidade de saída deve enviar resultados.
- **Unidade de armazenamento secundário:** O “depósito de longo prazo”. Armazena programas e dados de forma *persistente* (mantida mesmo com o computador desligado), em discos rígidos, CDs, DVDs, pendrives etc.

Boa prática de programação

Memorize as seis unidades lógicas e suas funções. Esse modelo conceitual é independente do hardware físico e será útil durante todo o curso, pois entender *onde* os dados vivem (entrada → memória → ALU/CPU → saída/disco) é essencial para escrever programas eficientes em C.

Item (e) – Os três tipos de linguagem de programação

Resposta detalhada

Resposta: linguagens de máquina, linguagens simbólicas (assembly) e linguagens de alto nível

Explicação passo a passo:

A Seção 1.6 apresenta a evolução histórica das linguagens de programação em três gerações:

- Linguagens de máquina:** O “idioma nativo” do hardware. Consistem em sequências de números binários (0s e 1s) que a CPU entende diretamente. São dependentes de máquina – um programa escrito para um tipo de CPU não funciona em outro. Extremamente difíceis para seres humanos lerem e escreverem.
- Linguagens simbólicas (assembly):** Substituem as sequências numéricas por abreviações mnemônicas (como `load`, `add`, `store`). Programas chamados *assemblers* (montadores) traduzem essas abreviações para linguagem de máquina. Ainda dependentes de

- máquina, porém muito mais legíveis que a linguagem de máquina pura.
- c) **Linguagens de alto nível:** Usam palavras próximas ao inglês e notação matemática convencional (como `valorTotal = salario + horaExtra;`). Uma única instrução de alto nível pode gerar dezenas de instruções de máquina. Programas chamados *compiladores* fazem essa tradução. **C é uma linguagem de alto nível.**

Item (f) – Programas que traduzem linguagem de alto nível para linguagem de máquina

Resposta detalhada

Resposta: compiladores

Explicação passo a passo:

Da Seção 1.6: programas tradutores chamados **compiladores** convertem os programas escritos em linguagem de alto nível (como C) para a linguagem de máquina que o computador pode executar.

É importante distinguir dois tipos de tradutores:

- **Compiladores:** Traduzem o programa *inteiro* de uma vez, gerando um arquivo executável (código objeto). A execução posterior é rápida.
- **Interpretadores:** Traduzem e executam o programa *linha por linha*, sem gerar um executável separado. São mais lentos na execução, mas eliminam a etapa de compilação.

Em C, usamos *compiladores* (como o `gcc` no Linux ou o Visual C++ no Windows).

Item (g) – O sistema operacional associado ao desenvolvimento de C

Resposta detalhada

Resposta: UNIX

Explicação passo a passo:

Da Seção 1.7: C foi criada por **Dennis Ritchie** no Bell Laboratories, em 1972, originalmente implementada em um computador DEC PDP-11. Sua principal motivação foi o desenvolvimento do **sistema operacional UNIX**. C e UNIX cresceram juntos: C tornou-se a linguagem de implementação do UNIX e ganhou popularidade com ele.

Hoje, praticamente todos os sistemas operacionais modernos – Linux, macOS, iOS, Android – têm raízes no UNIX e foram escritos em C e/ou C++. Isso ilustra o alcance e a longevidade de C como linguagem de sistemas.

Item (h) – O recurso da linguagem Ada para paralelismo

Resposta detalhada

Resposta: **multitarefa** (do inglês *multitasking*)

Explicação passo a passo:

Da Seção 1.11: a linguagem **Ada** foi desenvolvida pelo Departamento de Defesa dos EUA (DoD) e possui um recurso chamado **multitarefa**, que permite que os programadores especifiquem que muitas atividades devem ocorrer simultaneamente (em paralelo).

A Ada recebeu seu nome em homenagem a **Lady Ada Lovelace**, filha do poeta Lord Byron, reconhecida como a primeira programadora de computadores da história. Note que, embora a multitarefa não faça parte do C padrão (ANSI C), ela está disponível por meio de bibliotecas complementares, como a POSIX Threads (**pthread**s).

2. Exercício 1.2 – Preenchimento de Lacunas: Ambiente de Desenvolvimento em C

Enunciado

Preencha os espaços em cada uma das sentenças a respeito do ambiente C:

- Programas em C são normalmente digitados em um computador utilizando-se um programa _____.
- Em um sistema C, um programa _____ é executado automaticamente antes que se inicie a fase de tradução.
- Os tipos mais comuns de diretivas de pré-processador são utilizados para _____ e _____.
- O programa _____ combina a saída do compilador com várias funções de biblioteca para produzir uma imagem executável.
- O programa _____ transfere a imagem executável do disco para a memória.
- Para carregar e executar o programa compilado mais recentemente em um sistema Linux, digite _____.

Item (a) – O programa usado para digitar código C

Resposta detalhada

Resposta: **editor** (programa editor)

Explicação passo a passo:

A **Fase 1** do desenvolvimento de um programa em C, descrita na Seção 1.14 (Figura 1.1), consiste em *editar* o arquivo fonte. Um **programa editor** é utilizado para criar e modificar o código-fonte.

Exemplos de editores amplamente usados:

- No Linux: **vi**, **vim**, **emacs**, **nano**, **gedit**
- Em ambientes integrados (IDEs): o editor já vem embutido (ex.: Microsoft Visual Studio, Eclipse, Code::Blocks)

Os arquivos de código-fonte C devem ter extensão **.c** (por exemplo, **bemvindo.c**, **calculadora.c**).

Item (b) – O programa executado automaticamente antes da compilação

Resposta detalhada

Resposta: **pré-processador**

Explicação passo a passo:

Na **Fase 2** (descrita na Seção 1.14), antes que o compilador propriamente dito inicie a tradução do código C para linguagem de máquina, um programa especial chamado **pré-processador C** é executado *automaticamente*.

O pré-processador obedece a comandos especiais chamados **diretivas de pré-processador**, que começam com o caractere `#`. O pré-processador prepara o código-fonte para a compilação, realizando as manipulações indicadas por essas diretivas.

Boa prática de programação

Em um sistema C típico, o programador não precisa invocar o pré-processador manualmente. Ao executar o comando de compilação (`cc` ou `gcc`), o pré-processador é acionado automaticamente como primeira etapa.

Item (c) – Os usos mais comuns das diretivas de pré-processador

Resposta detalhada

Resposta: incluir outros arquivos no arquivo a ser compilado e substituir símbolos especiais por texto de programa

Explicação passo a passo:

As **diretivas de pré-processador** mais comuns, conforme a Seção 1.14, servem para:

- a) **Incluir outros arquivos:** A diretiva `#include` insere o conteúdo de outro arquivo no arquivo a ser compilado. Por exemplo:

```
#include <stdio.h>  /* inclui funcoes de entrada/
                     saida */
```

Isso é essencial para usar funções da biblioteca-padrão de C, como `printf` e `scanf`.

- b) **Substituições de texto (macros):** A diretiva `#define` cria *macros* – símbolos que serão substituídos por um valor ou trecho de código antes da compilação. Por exemplo:

```
#define PI 3.14159265
```

Toda ocorrência de `PI` no código será substituída por `3.14159265` antes que o compilador processe o arquivo.

Uma discussão detalhada de todas as diretivas do pré-processador é apresentada no Capítulo 13 do livro.

Item (d) – O programa que combina código objeto com bibliotecas

Resposta detalhada

Resposta: editor de ligação (ou *linker*)

Explicação passo a passo:

A **Fase 4** do processo é chamada de *ligação* (*linking*). O **editor de ligação** (ou *linker*) resolve as referências externas presentes no código objeto gerado pelo compilador.

Quando o compilador traduz seu código C para código objeto, ele deixa “lacunas” para as funções que você usou, mas que estão definidas em outros arquivos (por exemplo, na biblioteca-padrão de C). O *linker* preenche essas lacunas, combinando o código objeto do seu programa com o código das funções de biblioteca, produzindo uma **imagem executável** completa – um arquivo que pode ser executado pelo sistema operacional.

No Linux, tanto a compilação quanto a ligação são feitas pelo comando `gcc`:

```
gcc bemvindo.c    /* compila E liga, gerando a.out */
```

Item (e) – O programa que transfere a imagem executável para a memória

Resposta detalhada

Resposta: **carregador** (*loader*)

Explicação passo a passo:

A **Fase 5** é a *carga* (*loading*). O **carregador** (*loader*) pega a imagem executável armazenada no disco e a transfere para a **memória primária** (RAM), deixando-a pronta para ser executada pela CPU.

Além do executável principal, o carregador também carrega os componentes adicionais das **bibliotecas compartilhadas** (*shared libraries*) necessários ao programa. Em sistemas modernos, o carregador é chamado automaticamente pelo sistema operacional quando o usuário solicita a execução do programa.

Item (f) – Comando para executar o programa em Linux

Resposta detalhada

Resposta: `./a.out`

Explicação passo a passo:

Na **Fase 6** (execução), para carregar e executar o programa mais recentemente compilado em um sistema Linux, digita-se no terminal:

```
./a.out
```

Por que `./a.out`? Vamos entender cada parte:

- `./` → indica ao sistema operacional que o executável está no *diretório atual* (o “.” representa o diretório corrente e “/” é o separador de caminho no Linux).
- `a.out` → é o nome padrão do arquivo executável gerado pelo `gcc` quando nenhum nome específico é informado.

Você pode especificar um nome diferente para o executável usando a opção `-o`:

```
gcc bemvindo.c -o bemvindo    /* gera executavel chamado  
    "bemvindo" */  
./bemvindo                    /* executa o programa */
```

Erro comum de programação

Os comandos do Linux diferenciam maiúsculas de minúsculas (*case-sensitive*). Digitar ./A.OUT ou ./A.out resultará em erro, pois o arquivo gerado pelo gcc é a.out (letras minúsculas).

Visão Geral: O Ciclo de Desenvolvimento de Programas em C

Para consolidar os conceitos abordados no Exercício 1.2, apresentamos abaixo um resumo visual do ciclo completo de desenvolvimento de um programa em C (Figura 1.1 do livro):

As Seis Fases do Desenvolvimento em C

- Fase 1 Edição** → O programador cria o arquivo `.c` com um **editor de texto**.
- Fase 2 Pré-processamento** → O **pré-processador** executa as diretivas `#include`, `#define`, etc.
- Fase 3 Compilação** → O **compilador** traduz o código C para código objeto (linguagem de máquina).
- Fase 4 Ligação (Linking)** → O **editor de ligação** combina o código objeto com as bibliotecas, gerando a imagem executável (`a.out`).
- Fase 5 Carregamento (Loading)** → O **carregador** transfere o executável do disco para a memória RAM.
- Fase 6 Execução** → A **CPU** executa o programa, instrução por instrução.

Dica de portabilidade

Em sistemas Linux, todo esse processo (Fases 2 a 4) é normalmente ativado por um único comando: `gcc nome.c`. A simplicidade do processo é uma das razões pelas quais C permanece como a linguagem preferida para programação de sistemas em múltiplas plataformas.

Resumo das Respostas dos Exercícios de Autorrevisão

Gabarito Rápido – Capítulo 1

Exercício 1.1:

- a) Apple
- b) IBM Personal Computer (IBM PC)
- c) programas
- d) unidade de entrada, unidade de saída, unidade de memória, unidade lógica e aritmética (ALU), unidade central de processamento (CPU), unidade de armazenamento secundário
- e) linguagens de máquina, linguagens simbólicas (assembly), linguagens de alto nível
- f) compiladores
- g) UNIX
- h) multitarefa

Exercício 1.2:

- a) editor (programa editor)
- b) pré-processador
- c) incluir outros arquivos no arquivo a ser compilado; substituir símbolos especiais por texto de programa
- d) editor de ligação (*linker*)
- e) carregador (*loader*)
- f) `./a.out`

Boa prática de programação

Após verificar suas respostas, retorne ao Capítulo 1 e releia os trechos referentes aos itens que você errou. A leitura direcionada, motivada por uma dificuldade específica, é muito mais eficaz do que a leitura passiva e linear. Esse é o espírito da abordagem *incremental* Deitel.